

Threads problems.

18. • convert lowercase letters from cmd line to uppercase.
• print uppercase result.

#include...

```
typedef struct {
    char *word;
    int id;
} ThreadData;
```

```
void* worker(void* arg) {
    ThreadData *data = (ThreadData*) arg;
    int len = strlen(data->word);
    for (int i=0; i<len; i++) {
        data->word[i] = toupper(data->word[i]);
    }
    printf("Thread %d processed: %s\n", data->id, data->word);
    return NULL;
}
```

```
int main(int argc, char** argv) {
    if (argc < 2) {
        printf("Wrong usage\n");
        exit(1);
    }
    int m = argc - 1;
    pthread_t threads[m];
    ThreadData data[m];
    for (int i=0; i<m; i++) {
        data[i].word = argv[i+1];
        data[i].id = i;
        pthread_create(&threads[i], NULL, worker, &data[i]);
    }
    for (int i=0; i<m; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("\nFinal uppercase strings:\n");
    for (int i=1; i<argc; i++) {
        printf("%s\n", argv[i]);
    }
    return 0;
}
```

distinct for loop!

- 12a.)
- read matrix of int from file
 - create as many threads as rows in the matrix
 - each thread $\rightarrow$ sum on the row.
 - main process waits for threads to finish, then prints sums.
 - each thread computes its own sum

#include ...

typedef struct { //each thread has...

```
int id;  
int *row;  
int cols;  
int sum;
```

} ThreadData;

void* worker(void* arg) {

ThreadData *data = (ThreadData*) arg;

data $\rightarrow$ sum = 0;

```
for(int j=0; j<data  $\rightarrow$  cols; j++) {  
    data  $\rightarrow$  sum += data  $\rightarrow$  row[j];  
}
```

```
return NULL;  
}
```

}

int main(int argc, char** argv) {

if(argc != 2) {

printf("Wrong usage \n");

exit(1);

}

FILE *f = fopen(argv[1], "r");

if(f == NULL) { perror("fopen"); exit(1); }

int rows, cols;

fscanf(f, "%d %d", &rows, &cols);

int** matrix = malloc(rows * sizeof(int*));

```
for(int i=0; i<rows; i++) {
```

matrix[i] = malloc(cols * sizeof(int));

}

```
for(int i=0; i<rows; i++) {
```

```
    for(int j=0; j<cols; j++) {
```

fscanf(f, "%d", &matrix[i][j]);

```
    }
```

```
    }
```

pthread_t threads[rows];

ThreadData data[rows];

```
for(int i=0; i<rows; i++) {
```

data[i].id = i;

data[i].row = matrix[i];

data[i].cols = cols;

```
pthread_create(&threads[i], NULL, worker, &data[i]);
```

}

#1.

- read matrix of int from file
- create as many threads as rows
- all threads contribute to the same shared variable.

```
#include <stdio.h>
```

```
int total = 0;
```

```
pthread_mutex_t mymutex = PTHREAD_MUTEX_INITIALIZER;
```

```
typedef struct {
```

```
    int id;
```

```
    int *row;
```

```
    int cols;
```

```
} ThreadData;
```

```
void* worker(void* arg) {
```

```
    ThreadData* data = (ThreadData*) arg;
```

```
    int local_sum = 0;
```

```
    for (int j = 0; j < data->cols; j++) {
        local_sum += data->row[j];
    }
```

```
    pthread_mutex_lock(&mymutex);
```

```
    total += local_sum;
```

```
    pthread_mutex_unlock(&mymutex);
```

```
    printf("Thread %d added %d\n", data->id, local_sum);
```

```
    return NULL;
```

```
}
```

```
int main(int argc, char **argv) {
```

```
    if (argc != 2) {
```

```
        printf("Wrong usage\n");
```

```
        exit(1);
```

```
    }
```

```
    FILE *f = fopen(argv[1], "r");
```

```
    if (f == NULL) {
```

```
        perror("fopen");
```

```
        exit(1);
```

```
    }
```

```
    int rows, cols;
```

```
    fscanf(f, "%d %d", &rows, &cols);
```

```
    int **matrix = malloc(rows * sizeof(int*));
```

```
    for (int i = 0; i < rows; i++) {
```

```
        matrix[i] = malloc(cols * sizeof(int));
```

```
    }
```

```
    for (int i = 0; i < rows; i++) {
```

```
        for (int j = 0; j < cols; j++) {
```

```
            fscanf(f, "%d", &matrix[i][j]);
```

```
        }
```

```
    }
```



```

fclose(f);
pthread_t threads[rows];
ThreadData data[rows];
for (int i=0; i<rows; i++) {
    data[i].id = i;
    data[i].row = matrix[i];
    data[i].cols = cols;
    pthread_create(&threads[i], NULL, worker, &data[i]);
}
for (int i=0; i<rows; i++) {
    pthread_join(threads[i], NULL);
}
printf("\n Total matrix sum = %d\n", total);
for (int i=0; i<rows; i++) {
    free(matrix[i]);
}
free(matrix);
pthread_mutex_destroy(&mutex);
return 0;
}

```

- 15.
- receive strings as cmd-line args
 - for each string(word), compute
 - ◀ mr digits
 - ◀ mr letters
 - ◀ mr special characters
 - main program prints totals.
 - total mr. digits
 - total mr. letters
 - ...

```

#include ...
typedef struct {
    int id;
    char * word;
    int letters;
    int digits;
    int special;
} ThreadData;

```

```

void * worker(void * arg) {
    ThreadData * data = (ThreadData *) arg;
    data->letters = 0;
    data->digits = 0;
    data->special = 0;
    for (int i=0; data->word[i] != '\0'; i++) {
        char c = data->word[i];
        if (isalpha(c)) {
            data->letters++;
        }
        else if (isdigit(c)) {
            data->digits++;
        }
        else {
            data->special++;
        }
    }
    printf("Thread %d processed %s\n", data->id, data->word);
    return NULL;
}

```



```
Display_m(int argc, char **argv) {
    if (argc < 2) {
        printf("Wrong input\n");
        exit(1);
    }
```

```
    int m = argc - 1;
```

```
    pthread_t threads[m];
```

```
    ThreadData data[m];
```

```
    for (int i = 0; i < m; i++) {
```

```
        data[i].id = i;
```

```
        data[i].word = argv[i+1];
```

```
        pthread_create(&threads[i], NULL, worker, &data[i]);
    }
```

```
    for (int i = 0; i < m; i++) {
```

```
        pthread_join(threads[i], NULL);
    }
```

```
    int total_letters = 0;
```

```
    int total_digits = 0;
```

```
    int total_special = 0;
```

```
    for (int i = 0; i < m; i++) {
```

```
        total_letters += data[i].letters;
```

```
        total_digits += data[i].digits;
```

```
        total_special += data[i].special;
    }
```

```
    printf...
```

```
    return 0;
```

```
}
```

- 16.
- read int as cmd-line arg.
 - freq. vector for ~~each~~ all digits $\Rightarrow$ mutex
 - thread for each arg (^{word})
 - each thread counts the nr. of occurrences of each digit.

```
#...
```

```
int freq[10] = {0};
```

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

```
typedef struct {
```

```
    int id;
```

```
    int number;
```

```
} ThreadData;
```


void * worker(void * arg) {

ThreadData * data = (ThreadData *) arg;

int x = data -> number;

if (x == 0) {

pthread_mutex_lock(&mutex);

freq[0]++;

pthread_mutex_unlock(&mutex);

}

while (x > 0) {

int digit = x % 10;

pthread_mutex_lock(&mutex);

freq[digit]++;

pthread_mutex_unlock(&mutex);

}

x = x / 10;

printf("Thread %d processed nr %d\n", data -> id, data -> number);

return NULL;

}

int main(int argc, char **argv) {

if (argc < 2) {

printf("Wrong usage\n");

exit(1);

}

int m = argc - 1;

pthread_t threads[m];

ThreadData data[m];

for (int i = 0; i < m; i++) {

data[i].id = i;

data[i].number = atoi(argv[i+1]);

pthread_create(&threads[i], NULL, worker, &data[i]);

}

for (int i = 0; i < m; i++) {

pthread_join(threads[i], NULL);

}

printf---

for (---) {

printf("%d -> %d\n", i, freq[i]);

}

pthread_mutex_destroy(&mutex);

return 0;

}

- 4 report
- 2 threads
 - 1 thread → generate even nr → append to array → global variable (here)
 - other thread → generate odd nr → ... → param (task)
 - alternate adding value to array ⇒ COND. VARIABLES.

#include ...

int *array;

int m;

int pos = 0;

int turn = 0; // shared control variable

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

pthread_cond_t cond = PTHREAD_COND_INITIALIZER; // wake the thread whose turn comes next

typedef struct {

int id;

int parity; // 0 → even generator thread; 1 → odd generator thread

} ThreadData;

void* worker(void* arg) {

ThreadData *data = (ThreadData*) arg;

while(1) {

pthread_mutex_lock(&mutex);

while (turn != data->parity && pos < m) {

pthread_cond_wait(&cond, &mutex);

}

if (pos >= m) {

pthread_cond_signal(&cond);

pthread_mutex_unlock(&mutex);

break;

}

int value;

if (data->parity == 0) {

value = (rand() % 100 * 2);

} else {

value = (rand() % 100) * 2 + 1;

}

array[pos] = value;

printf("Thread %d added %d at position %d\n", data->id, value, pos);

pos++;

turn = 1 - turn;

pthread_cond_signal(&cond); // wake the other thread

pthread_mutex_unlock(&mutex); // allow other thread to enter

}

return NULL;

}


```
int main() {
```

```
    srand(time(NULL));
```

```
    printf("Give m: ");
```

```
    scanf("%d", &m);
```

```
    array = malloc(m * sizeof(int));
```

```
    pthread_t threads[2];
```

```
    ThreadData data[2];
```

```
    data[0].id = 0;
```

```
    data[0].parity = 0;
```

```
    data[1].id = 1;
```

```
    data[1].parity = 1;
```

```
    pthread_create(&threads[0], NULL, worker, &data[0]);
```

```
    pthread_create(&threads[1], NULL, worker, &data[1]);
```

```
    pthread_join(threads[0], NULL); // wait until even finishes
```

```
    pthread_join(threads[1], NULL); // " " "odd" " " "
```

```
    printf("In Final Array: \n");
```

```
    for(int i=0; i<m; i++) {
```

```
        printf("%d", array[i]);
```

```
    }
```

```
    free(array);
```

```
    pthread_mutex_destroy(&mutex);
```

```
    pthread_cond_destroy(&cond);
```

```
    return 0;
```

```
}
```


summary 5

```
int x;  
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER; // create initially-unlocked mutex.  
void *f(void *arg) {  
    int v; // local var.  
    v = *(int *)arg; // read int stored at addr. arg  
    pthread_mutex_lock(&m); // lock mutex  
    x++; // safely increment  
    pthread_mutex_unlock(&m); // unlock mutex  
    printf("Inside thread, arg received: %d\n", v);  
    return NULL;  
}  
int main() {  
    pthread_t t; // real thread id  
    pthread_create(&t, NULL, f, (void *) &t); // addr. of t.  
    pthread_mutex_lock(&m);  
    x++;  
    pthread_mutex_unlock(&m);  
    printf("Created main thread with id %d\n", t);  
    pthread_join(t, NULL);  
    printf("Done thread, main done, x: %d\n", x);  
    pthread_mutex_destroy(&m);  
    return 0;  
}
```



```

for (int i=0; i<rows; i++){
    pthread_join(threads[i], NULL);
}
for (int i=0; i<rows; i++){
    pthread
    printf("Row %d sum = %d\n", data[i].id, data[i].sum);
}
for (int i=0; i<rows; i++){
    free(matrix[i]);
}
free(matrix);
return 0;
}

```

Seminar ex1 - create 5 threads for which print a message...

```

#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
void *PrintHello(void *threadid){
    long tid = (long)threadid; ← reinterpret back into a number
    printf("Hello world! It's thread %ld\n", tid);
    pthread_exit(NULL);
}
int main(int argc, char *argv[]){
    pthread_t threads[NUM_THREADS];
    int rc; // return code
    long t;
    for (int t=0; t<NUM_THREADS; t++){
        printf("Creating thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        if (rc){
            printf("Error from pthread_create() is %d\n", rc);
            exit(-1);
        }
    }
    pthread_exit(NULL);
}

```

← "a dirty trick" from lecture.

include...

int m; // threads

int n; // checkpoints

#1. pthread_mutex_t *checkpoint_mutexes; // one mutex for each checkpoint (n)
pthread_barrier_t start_barrier; // all threads wait here before starting race

typedef struct {

int id;

} ThreadData;

void * worker(void* arg) {

ThreadData* data = (ThreadData*) arg;

pthread_barrier_wait(&start_barrier); // thread pauses here until all arrive
for(int checkpoint = 0; checkpoint < n; checkpoint++) {

pthread_mutex_lock(&checkpoint_mutexes[checkpoint]);

usleep(10000 + rand() % 10001);

printf("Thread %d passed checkpoint %d\n", data->id, checkpoint);

pthread_mutex_unlock(&checkpoint_mutexes[checkpoint]);

}

return NULL;

}

int main(int argc, char **argv) {

if(argc != 3) {

printf("Wrong\n");

exit(1);

}

m = atoi(argv[1]);

n = atoi(argv[2]);

srand(time(NULL));

pthread_t threads[m];

ThreadData data[m]; // one struct per thread

checkpoint_mutexes = malloc(m * sizeof(pthread_mutex_t)); // allocate m mutexes dynamically.

for(int i = 0; i < m; i++) { // initialise each checkpoint mutex

pthread_mutex_init(&checkpoint_mutexes[i], NULL);

pthread_barrier_init(&start_barrier, NULL, m); // barrier waits for m threads.

for(int i = 0; i < m; i++) { // create threads

data[i].id = i;

pthread_create(&threads[i], NULL, worker, &data[i]);

for(int i = 0; i < m; i++) { // wait for all threads to finish

pthread_join(threads[i], NULL);

}

for(int i = 0; i < m; i++) { // destroy mutexes

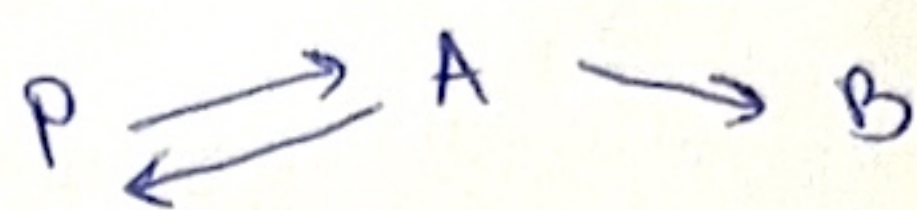
pthread_mutex_destroy(&checkpoint_mutexes[i]);

}

pthread_barrier_destroy(&start_barrier);

free(checkpoint_mutexes);

return 0;



process → children: A, B every time a string is read from keyboard.

- #1 process A:
- receives string from parent, checks uppercase / lowercase letter
 - uppercase ⇒ add to U → send to B
 - lowercase ⇒ add to L → send to B
 - not letter ⇒ increment S. → send S to parent → parent prints S

- Process B:
- receives L, U from A
 - sum up sizes → send to print to screen

PIPES

#include...

int main()

int p2a[2], a2p[2], a2b[2];

pipe(p2a); pipe(a2p); pipe(a2b);

if (fork() == 0) // A

close(a2p[0]); close(p2a[1]);

close(a2b[0]);

char s[256], l[256], u[256];

int m;

while(1)

if (read(p2a[0], &m, sizeof(int)) <= 0)

break;

if (read(p2a[0], &s, m) <= 0)

break;

int S=0, j=0, k=0;

for (int i=0; i<m; i++)

if (s[i] >= 'a' && s[i] <= 'z')

l[j++] = s[i];

else if (s[i] >= 'A' && s[i] <= 'Z')

u[k++] = s[i];

else S++;

write(a2p[1], &S, sizeof(int));

write(a2b[1], &j, sizeof(int));

if (j > 0)

write(a2b[1], l, j * sizeof(char));

write(a2b[1], &k, sizeof(int));

if (k > 0)

write(a2b[1], u, k * sizeof(char));

}

close(a2p[1]);

close(p2a[0]);

close(a2b[1]);

exit(0);

}

if (fork() == 0) // B

B: { a2b[0]

A: { a2p[1] ✓
p2a[0] ✓
a2b[1] ✓


```

close (pza[0]);
close (pza[1]);
close (azp[0]);
....

```

```

char l[256], u[256];

```

```

int j, k;

```

```

while (1) {

```

```

    if (read (azb[0], &j, sizeof (int)) <= 0)
        break;

```

```

    if (j > 0)

```

```

        if (read (azb[0], l, j * sizeof (char)) <= 0)
            break;

```

```

    if (read (azb[0], &k, sizeof (int)) <= 0)
        break;

```

```

    if (k > 0)

```

```

        if (read (azb[0], u, k * sizeof (char)) <= 0)
            break;

```

```

    printf ("Sizes are: %d \n", j+k);

```

```

}
close (azb[0]);

```

```

exit(0);

```

```

}
close (pza[0]);
close ....

```

// parent P:

```

{ pza[1]
  azp[0]

```

```

char S[256];

```

```

while (1) {

```

```

    printf ("Give string of x: ");

```

```

    scanf ("%s", S);

```

```

    if (strcmp (S, "x") == 0) break;

```

```

    int m = strlen (S);

```

```

    write (pza[1], &m, sizeof (int));

```

```

    write (pza[1], S, m * sizeof (char));

```

```

    int S;

```

```

    read (azp[0], &S, sizeof (int));

```

```

    printf ("S is: %d \n", S);

```

```

}

```

```

close (pza[1]);

```

```

close (azp[0]);

```

```

wait(0);

```

```

wait(0);

```

```

return 0;

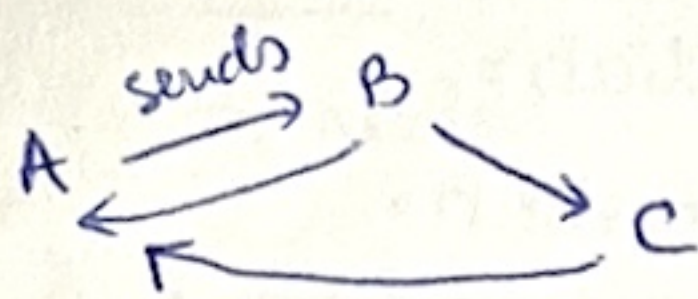
```

```

}

```


Test 3 Prep - Processes



#1 problem 1

Process A - read lowercase strings until x, send them to B, print sum from C
 Process B - receive from A, initiate vector with 11 zeroes, check digit / non-digit, increment index of vector, send vector to A and C

Process C - receive from B, sum up values, send to A

FIFO

a.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

int main() {
    mkfifo("a2b", 0600);
    mkfifo("b2a", 0600);
    mkfifo("c2a", 0600);

    int a2b = open("a2b", O_WRONLY);
    int b2a = open("b2a", O_RDONLY);
    int c2a = open("c2a", O_RDONLY);

    if (a2b < 0 || b2a < 0 || c2a < 0) {
        perror("open");
        exit(1);
    }

    char s[100];
    int m, v[11];
    int sum;
    while (1) {
        printf("Give strings (x to exit): ");
        scanf("%s", s); // one word
        m = strlen(s); // length of word
        if (strcmp(s, "x") == 0)
            break;

        write(a2b, &m, sizeof(int));
        write(a2b, s, m);
        read(b2a, v, 11 * sizeof(int));
        read(c2a, &sum, sizeof(int));

        printf("Vector: ");
        for (int i = 0; i < 11; i++)
            printf("%d", v[i]);
        printf("\n");
        printf("Sum: %d\n", sum);
    }

    close(a2b); close(b2a); close(c2a); return 0;
}
  
```

b.c

```

#include ...
int main() {
    mkfifo("a2b", 0600);
    mkfifo("b2a", 0600);
    mkfifo("c2a", 0600);
    int a2b = open("a2b", O_RDONLY);
    int b2a = open("b2a", O_WRONLY);
    int c2a = open("b2c", O_WRONLY);

    if (a2b < 0 || b2a < 0 || c2a < 0) {
        perror("open");
        exit(1);
    }

    char s[100];
    int m;
    while (1) {
        if (read(a2b, &m, sizeof(int)) <= 0)
            break;
        if (read(a2b, s, m) <= 0)
            break;
        int v[11] = {0};
        for (int i = 0; i < m; i++) {
            if (s[i] >= '0' && s[i] <= '9') // digit
                v[s[i] - '0']++;
            else
                v[10]++; // non-digit (end)
        }
        write(b2a, v, 11 * sizeof(int));
        write(b2c, v, 11 * sizeof(int));
    }

    close(a2b);
    close(b2a);
    close(b2c);
    return 0;
}
  
```


c.c

#include...

```
int main() {
    mkfifo("b2c", 0600);
    mkfifo("c2a", 0600);
    int b2c = open("b2c", O_RDONLY);
    int c2a = open("c2a", O_WRONLY);
    if (b2c < 0 || c2a < 0) {
        perror("open");
        exit(1);
    }
    int v[11];
    while(1) {
        if (read(b2c, v, 11 * sizeof(int)) <= 0)
            break;
        int sum = 0;
        for (int i = 0; i < 11; i++)
            sum += v[i];
        write(c2a, &sum, sizeof(int));
    }
    close(b2c);
    close(c2a);
    return 0;
}
```

* //parent A

```
close(a2b[0]); ...
char s[100];
int m, v[11], sum;
while(1) {
    printf("Insert string (x to exit): ");
    scanf("%s", s);
    m = strlen(s);
    if (strcmp(s, "x") == 0) break;
    write(a2b[1], &m, sizeof(int));
    write(a2b[1], s, m * sizeof(char));
    read(b2a[0], v, 11 * sizeof(int));
    read(c2a[0], &sum, sizeof(int));
    printf("vector: ");
    for (int i = 0; i < 11; i++)
        printf("%d", v[i]);
    printf("\n");
    printf("sum: %d\n", sum);
}
close(a2b[1]); close(b2a[0]); close(c2a[0]);
wait(0); wait(0);
return 0;
}
```

A: { a2b[1]
b2a[0]
c2a[0]

#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

int a2b[2];

int b2a[2];

int b2c[2];

int c2a[2];

pipe(a2b);

pipe(b2a);

pipe(b2c);

pipe(c2a);

if (fork() == 0) { // child C

close(b2c[1]); close(c2a[0]);

close(a2b[0]); close(a2b[1]);

close(b2a[0]); close(b2a[1]);

int v[11];

while(1) {

if (read(b2c[0], v, 11 * sizeof(int)) <= 0)

break;

int sum = 0;

for (int i = 0; i < 11; i++)

sum += v[i];

write(c2a[1], &sum, sizeof(int));

}

close(b2c[0]);

close(c2a[1]);

exit(0);

if (fork() == 0) { // child B

close(a2b[1]); close(b2a[0]);

close(b2c[0]); close(c2a[0]);

close(c2a[1]);

char s[100];

int m;

while(1) {

if (read(a2b[0], &m, sizeof(int)) <= 0)

break;

if (read(a2b[0], s, m * sizeof(char)) <= 0)

break;

int v[11] = {0};

for (int i = 0; i < m; i++) {

if (s[i] >= '0' && s[i] <= '9') // digit

v[s[i] - '0']++;

else

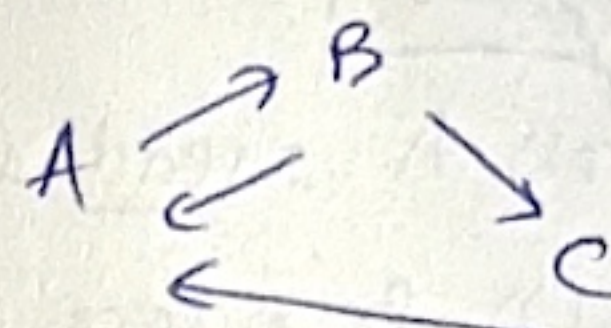
v[10]++; // non-digit

}

write(b2a[1], v, 11 * sizeof(int));

write(b2a[1], v, 11 * sizeof(int));

* close(a2b[0]); close(b2c[1]); close(b2a[1]); exit(0);

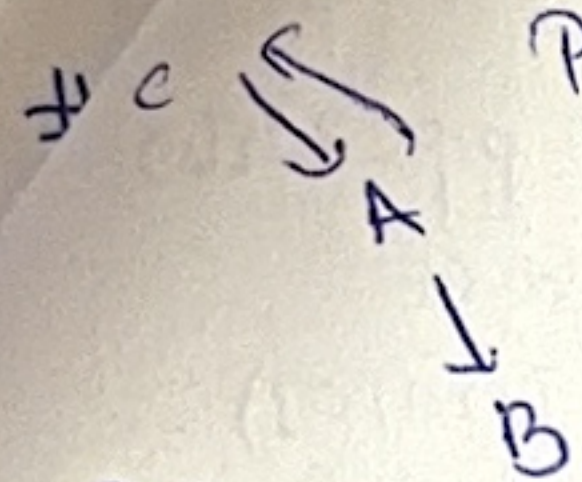


C: { b2c[0]
c2a[1]

B: { a2b[0]
b2c[1]
b2a[1]

Process A • receive string from C, compute v, send vector to B, print frequency

Process B • receive vector from A, compute total of lowercase letters, prints the sum.



a.c

```

#include <sys/stat.h>
int main() {
    int c2a, a2b, a2c;
    mkfifo("c2a", 0600);
    mkfifo("a2c", 0600);
    mkfifo("a2b", 0600);
    c2a = open("c2a", O_RDONLY);
    a2b = open("a2b", O_WRONLY);
    a2c = open("a2c", O_WRONLY);
    if (c2a < 0 || a2b < 0 || a2c < 0) {
        perror("open");
        exit(1);
    }
    int m;
    char s[100];
    while(1) {
        if (read(c2a, &m, sizeof(int)) <= 0)
            break;
        if (read(c2a, s, m * sizeof(char)) <= 0)
            break;
        int v[26] = {0};
        for (int i = 0; i < m; i++) {
            if (s[i] >= 'a' & s[i] <= 'z')
                v[s[i] - 'a']++;
        }
        write(a2b, v, 26 * sizeof(int));
        write(a2c, v, 26 * sizeof(int));
    }
    close(c2a);
    close(a2b);
    close(a2c);
    return 0;
}
  
```

b.c

```

#include ...
int main() {
    int a2b;
    mkfifo("a2b", 0600);
    a2b = open("a2b", O_RDONLY);
    if (a2b < 0) {
        perror("open");
        exit(1);
    }
    int v[26];
    for (int i = 0; i < 26; i++)
    while(1) {
        if (read(a2b, v, 26 * sizeof(int)) <= 0)
            break;
        int sum = 0;
        for (int i = 0; i < 26; i++)
            sum += v[i];
        printf("Sum is: %d\n", sum);
    }
    close(a2b);
    return 0;
}
  
```


c.c

#include ...

```
int main() {
    int cza, azc;
    mkfifo("cza", 0600);
    mkfifo("azc", 0600);
    cza = open("cza", O_WRONLY);
    azc = open("azc", O_RDONLY);
    if (cza < 0 || azc < 0) {
        perror("open");
        exit(1);
    }
    char s[100];
    int m, v[26];
    while (1) {
        printf("Insert string (x to exit): ");
        scanf("%s", s);
        if (strcmp(s, "x") == 0)
            break;
        m = strlen(s);
        write(cza, &m, sizeof(int));
        write(cza, s, m * sizeof(char));
        if (read(azc, n, 26 * sizeof(int)) < 0)
            break;
        for (int i = 0; i < 26; i++)
            printf("%c -> %d\n", 'a' + i, v[i]);
    }
    close(cza);
    close(azc);
    return 0;
}
```

PIPES | #include ...

```
int main() {
    int cza[2];
    int azc[2];
    int azb[2];
    pipe(cza); pipe(azc); pipe(azb);
    if (fork() == 0) { // child B
        close(azb[1]); close(cza[0]);
        int v[26];
        while (1) {
            if (read(azb[0], n, 26 * sizeof(int)) < 0) break;
            int sum = 0;
            for (int i = 0; i < 26; i++)
                sum += v[i];
            printf("Sum: %d\n", sum);
        }
        close(azb[0]);
        exit(0);
    }
    if (fork() == 0) { // child A
        close(...); ...
        int m;
        char s[100];
        while (1) {
            if (read(cza[0], &m, ... < 0)
                break;
            if (read(cza[0], s, m * ... < 0)
                break;
            int v[26] = {0};
            for (int i = 0; i < m; i++) {
                if ...
            }
        }
    }
}
```